ASSINGMENT--6 (RESPONSIBLE AI)

Task Description #1 (Privacy & Security in File Handling)

PROMPT: generate a Python script that stores user data (name, email, password) in a file. AND Revised version with encrypted password storage (e.g., hashing).

CODE:

```
def store_user_data(name, email, password):
    # Open the file in append mode
    with open('user_data.txt', 'a') as file:
        # Write the user data to the file
        file.write(f'Name: {name}, Email: {email}, Password: {password}\n')
#example  usage
if __name__ == "__main__":
    # Get user input
    name = input("Enter your name: ")
    email = input("Enter your email: ")
    password = input("Enter your password: ")

    # Store the user data
    store_user_data(name, email, password)

    print("User data stored successfully.")
#generate a Revised version with encrypted password storage (e.g., hashing).
# Python script to store user data (name, email, password) in a file with encrypted password storage
import hashlib
```

```
def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

def store_user_data(name, email, password):
    # Hash the password
    hashed_password = hash_password(password)

    # Open the file in append mode
    with open('user_data.txt', 'a') as file:
        # Write the user data to the file
        file.write(f'Name: {name}, Email: {email}, Password: {hashed_password}\n')

#example  usage
if __name__ == "__main__":
    # Get user input
    name = input("Enter your name: ")
    email = input("Enter your email: ")
    password = input("Enter your password: ")

    # Store the user data
    store_user_data(name, email, password)

    print("User data stored successfully.")
```

OUTPUT:

```
PS C:\Users\jeshwanth\Documents\.vscode> python -u "c:\Users\jeshwanth\Documents\.vscode\github\tempCodeRunnerFile.py"
Enter your name: jeshwanth
Enter your email: jeshwanth@gmail.com
Enter your password: jeshwanth@16
User data stored successfully.
Enter your name: jeshwanth
Enter your email: jeshwanth@gmail.com
Enter your password: jeshwanth@16
User data stored successfully.
PS C:\Users\jeshwanth\Documents\.vscode>
```

Task Description #2 – Fair Decision Logic

PROMPT: generate a scholarship eligibility checker based on academic score, family income, and location. Examine whether the logic unfairly favors or disadvantages any group.

CODE:

```
def check_scholarship_eligibility(academic_score, family_income, location):
    # Define eligibility criteria
    min_academic_score = 85
    max_family_income = 50000
    eligible_locations = ['urban', 'suburban']

    # Check academic score
    if academic_score < min_academic_score:
        return "Not eligible: Academic score is below the minimum requirement."

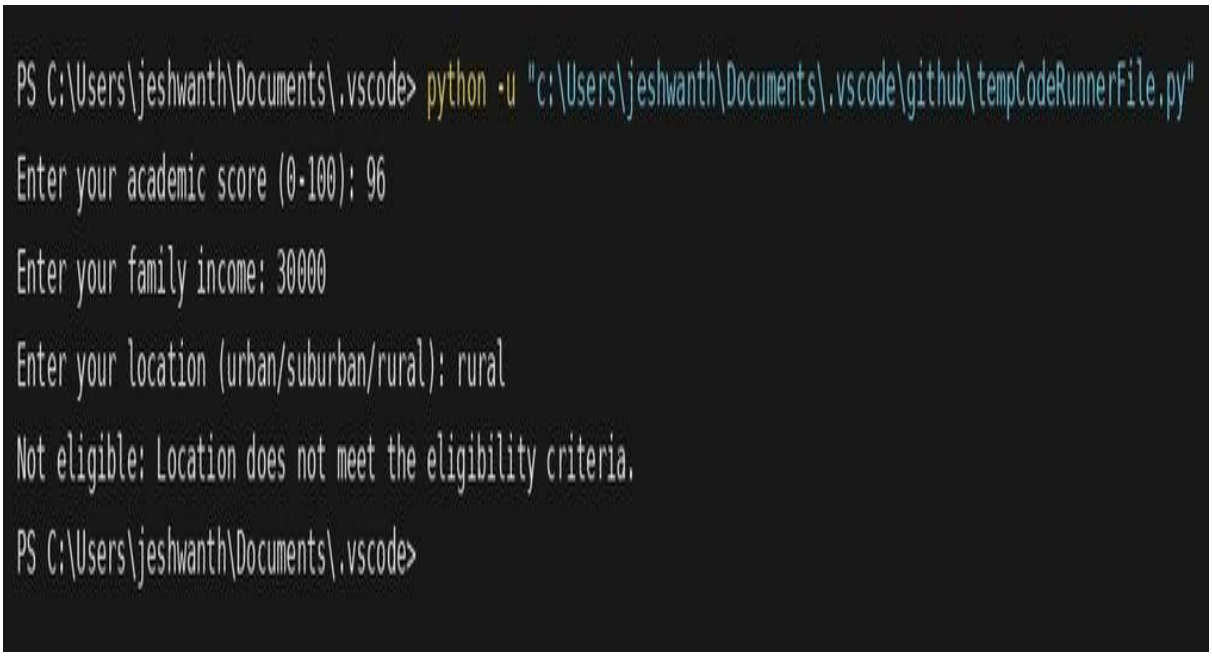
    # Check family income
    if family_income > max_family_income:
        return "Not eligible: Family income exceeds the maximum limit."
```

```
    # Check location
    if location.lower() not in eligible_locations:
        return "Not eligible: Location does not meet the eligibility criteria."

    return "Eligible for scholarship."
# Example usage
if __name__ == "__main__":
    # Get user input
    academic_score = float(input("Enter your academic score (0-100): "))
    family_income = float(input("Enter your family income: "))
    location = input("Enter your location (urban/suburban/rural): ")

    # Check scholarship eligibility
    result = check_scholarship_eligibility(academic_score, family_income, location)
    print(result)
```

OUTPUT:



Task Description #3 (Transparency in AI-Based Admission System)

PROMPT: generate a Python program that predicts whether a student should be selected or rejected for admission based on factors such as entrance exam score, academic marks, and other relevant admission criteria.

CODE:

```
def predict_admission(entrance_exam_score, academic_marks, extracurricular_activities):


    # Define admission criteria

    min_entrance_exam_score = 70

    min_academic_marks = 75

    extracurricular_weight = 10


    # Calculate total score

    total_score = entrance_exam_score + academic_marks + extracurricular_activities * extracurricular_weight


    # Determine admission decision

    if entrance_exam_score < min_entrance_exam_score:

        return "Rejected: Entrance exam score is below the minimum requirement."


    if academic_marks < min_academic_marks:

        return "Rejected: Academic marks are below the minimum requirement."


    if total_score >= 200:

        return "Selected for admission."

    else:
```

```
    return "Rejected: Total score does not meet the admission criteria."

# Example usage

if __name__ == "__main__":

  # Get user input

  entrance_exam_score = float(input("Enter your entrance exam score (0-100): "))

  academic_marks = float(input("Enter your academic marks (0-100): "))

  extracurricular_activities = int(input("Enter the number of extracurricular activities participated in: "))


  # Predict admission decision

  result = predict_admission(entrance_exam_score, academic_marks, extracurricular_activities)

  print(result)
```

OUTPUT:



Task Description #4 – Ethical Scoring System

PROMPT: Task: Generate an employee performance evaluation system using inputs such as project completion rate, teamwork score, and attendance. Analyze the scoring logic for any unethical weighting or hidden bias.

CODE:

```
def evaluate_employee_performance(project_completion_rate, teamwork_score, attendance):
```

```
    # Define performance criteria
    min_project_completion_rate = 70
    min_teamwork_score = 60
    min_attendance = 80

    # Calculate total score
    total_score = (project_completion_rate * 0.5) + (teamwork_score * 0.3) + (attendance
* 0.2)

    # Determine performance evaluation
    if project_completion_rate < min_project_completion_rate:
        return "Needs Improvement: Project completion rate is below the minimum
requirement."

    if teamwork_score < min_teamwork_score:
        return "Needs Improvement: Teamwork score is below the minimum requirement."

    if attendance < min_attendance:
        return "Needs Improvement: Attendance is below the minimum requirement."

    if total_score >= 75:
        return "Excellent Performance."
    elif total_score >= 60:
        return "Good Performance."
    else:
        return "Needs Improvement."
#example usage
if __name__ == "__main__":
    # Get user input
```

```
    project_completion_rate = float(input("Enter project completion rate (0-100): "))
    teamwork_score = float(input("Enter teamwork score (0-100): "))
    attendance = float(input("Enter attendance percentage (0-100): "))

    # Evaluate employee performance
    result = evaluate_employee_performance(project_completion_rate, teamwork_score, attendance)
    print(result)
```

OUTPUT:

```
PS C:\Users\jeshwanth\Documents\.vscode> python -u "c:\Users\jeshwanth\Documents\.vscode\github\tempCodeRunnerFile.py"
Enter project completion rate (0-100): 96
Enter teamwork score (0-100): 92
Enter attendance percentage (0-100): 88
Excellent Performance.
PS C:\Users\jeshwanth\Documents\.vscode>
```

Task Description #5 – Accessibility and Inclusiveness

PROMPT: generate a user feedback form application. Review whether the language and options provided are inclusive and accessible to diverse users.

CODE:

```
def user_feedback_form():
    # Collect user feedback
    print("Welcome to the User Feedback Form")
    name = input("Please enter your name: ")
    email = input("Please enter your email: ")
    feedback = input("Please provide your feedback: ")

    # Store feedback in a file
    with open('user_feedback.txt', 'a') as file:
```

```
        file.write(f'Name: {name}, Email: {email}, Feedback: {feedback}\n')


    print("Thank you for your feedback!")
#example usage
if __name__ == "__main__":
    user_feedback_form()
```

OUTPUT:

```
PS C:\Users\jeshwanth\Documents\.vscode> python -u "c:\Users\jeshwanth\Documents\.vscode\github\tempCodeRunnerFile.py"
Welcome to the User Feedback Form
Please enter your name: jeshwanth
Please enter your email: jeshwanth@gmail.com
Please provide your feedback: i am happy
Thank you for your feedback!
PS C:\Users\jeshwanth\Documents\.vscode>
```

Task Description #6 (Fairness in Loan Approval System)

PROMPT: generate a Python program that predicts whether a person should be approved or rejected for a loan based on information such as income, age, employment status, credit score, and loan amount.

CODE:

```
def predict_loan_approval(income, age, employment_status, credit_score, loan_amount):
    # Define loan approval criteria
    min_income = 30000
    min_age = 21
    min_credit_score = 650
    max_loan_amount = 50000


    # Check income
    if income < min_income:
        return "Rejected: Income is below the minimum requirement."
```

```
    # Check age
    if age < min_age:
        return "Rejected: Age is below the minimum requirement."

    # Check employment status
    if employment_status.lower() not in ['employed', 'self-employed']:
        return "Rejected: Employment status does not meet the eligibility criteria."

    # Check credit score
    if credit_score < min_credit_score:
        return "Rejected: Credit score is below the minimum requirement."

    # Check loan amount
    if loan_amount > max_loan_amount:
        return "Rejected: Loan amount exceeds the maximum limit."

    return "Approved for loan."
# Example usage
if __name__ == "__main__":
    # Get user input
    income = float(input("Enter your income: "))
    age = int(input("Enter your age: "))
    employment_status = input("Enter your employment status (employed/self-employed/unemployed): ")
    credit_score = int(input("Enter your credit score (300-850): "))
    loan_amount = float(input("Enter the desired loan amount: "))
```

```
# Predict loan approval
result = predict_loan_approval(income, age, employment_status, credit_score, loan_amount)
print(result)
```

OUTPUT:

